

Subject Name : Machine Learning

Unit No: 1

**Unit Name : Introduction to Machine Learning
& Data Preprocessing**

Faculty Name : Dr. Ritu Jain

Data Preprocessing in Machine Learning:

Handling Missing values

- The quality of the data and the amount of useful information that it contains are key factors that determine how well a machine learning algorithm can learn.
- Therefore, it is absolutely critical to ensure that we examine and preprocess a dataset before we feed it to a learning algorithm.
- Essential data preprocessing techniques help us to build good machine learning models



How is a Missing Value Represented in a Dataset?

- Missing values in a dataset can be represented in various ways, depending on the source of the data and the conventions used. Here are some common representations:
 - **NaN (Not a Number):** In many programming languages and data analysis tools, missing values are represented as NaN. This is the default for libraries like **Pandas** in Python.
 - **NULL or None:** In **databases** and some programming languages, missing values are often represented as NULL or None. For instance, in **SQL databases**, a missing value is typically recorded as NULL.
 - **Empty Strings:** Sometimes, missing values are denoted by empty strings (""). This is common in text-based data or CSV files where a field might be left blank.
 - **Special Indicators:** Datasets might use specific indicators like **-999**, **9999**, or other unlikely values to signify missing data. This is often seen in **older datasets** or specific industries where such conventions were established.
 - **Blanks or Spaces:** In some cases, particularly in **fixed-width text files**, missing values might be represented by spaces or blank fields.



Types of Missing Values

Understanding the types of missing values in datasets is crucial for effectively handling missing data and ensuring accurate analyses:

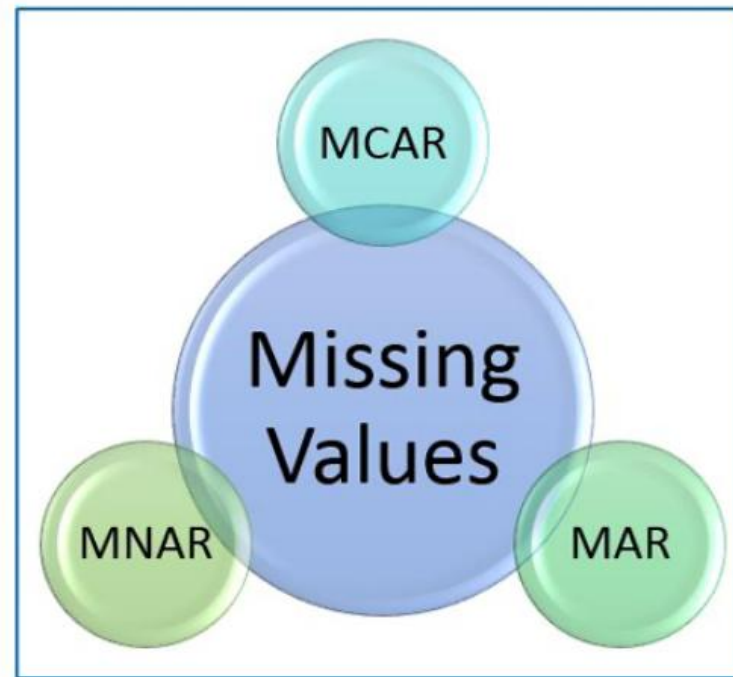


Figure 1 - Different Types of Missing Values in Datasets

Missing Completely At Random (MCAR)

- MCAR occurs when the probability of data being missing is uniform across all observations.
- There is no relationship between the missingness of data and any other observed or unobserved data within the dataset.
- This type of missing data is purely random and lacks any discernible pattern.
- **Example:**
 - Example: A sensor fails randomly, causing some temperature readings to be missing, unrelated to temperature or time.
 - Example: Participants randomly skip a few questions on a 15-question survey.
- Thus, the missingness is directly dependent on the very variable that is missing in the first place.



Missing At Random (MAR)

- ✓ MAR data occurs when the probability of data being missing depends only on the observed data and not on the missing data itself.
- ✓ In other words, the missingness can be explained by variables for which you have complete information.
- ✓ There is a pattern in the missing values, but this pattern can be explained by other observed variables.



- ✓ **Example:** For instance, in an academic performance survey, students with higher grades might be less likely to disclose information about the number of hours they study (due to competition, maybe).
- ✓ Suppose a survey is taken at a dermatology clinic where the gender and their skincare routine is asked. Assume that most of the females answer the survey whereas men are less likely to answer. So here, why the data is missing could be explained by the factor, that is gender. In this case, the missing data in the skincare routine column is



D.Y. PATIL
UNIVERSITY

NAVI MUMBAI

MAR

Missing Not At Random (MNAR)

- ✓ MNAR occurs when the missingness of data is related to the unobserved data itself, which is not included in the dataset.
- ✓ In this case, the missing values are not random and cannot be explained by the observed data. This could be a challenging case as possible reasons for the missingness are related to the unobserved data.
- ✓ **Example 1:** For instance, in a health survey, participants with very high stress levels might consciously choose not to disclose their stress level due to stigma, or fear of judgment. **As a result, the missing data on stress level is not random; it is influenced by the stress level itself.** So, in a way, the higher the stress level, the less likely one will disclose it, and the more likely the value will be missing from the collected dataset.
- **Example 2:** people having more income may refuse to share the exact information in a survey or questionnaire.



Example of missing data

School ID	Name	Address	City	Subject	Marks	Rank	Grade
101.0	A	123 Main St	Mumbai	Math	85.0	2	B
102.0	B	456 Oak Ave	Delhi	English	92.0	1	A
103.0	C	789 Pine Ln	Bengaluru	Science	78.0	4	C
NaN	D	101 Elm St	Chennai	Math	89.0	3	B
105.0	E	NaN	Kolkata	History	NaN	8	D
106.0	F	222 Maple Rd.	NaN	Math	95.0	1	A
107.0	G	444 Cedar Blvd	Pune	Science	80.0	5	C
108.0	H	555 Birch Dr	Jaipur	English	88.0	3	B



Reasons Behind Missing Values in the Dataset

- **Technical issues:** Failed data collection or errors during data transmission. Eg: IoT temperature sensor loses power for one hour, resulting in a null temperature value for that period.
- **Human errors:** Mistakes like incorrect data entry or oversights during data processing. Eg: Survey Non-Response
- **Privacy concerns:** Missing sensitive or personal information due to confidentiality policies. Eg: healthcare dataset removes SSN to comply with regulations, leaving SSN fields blank. In a financial survey, high-net-worth individuals refuse to disclose their "Total Assets" column.
- **Data processing issues:** Errors that occur during data preparation. Eg: Invalid Join, Invalid Data Type Conversion
- **Not Applicable / Design-Based Missingness:** certain measurements may not be applicable. Eg: Salary field for unemployed individuals

By identifying the reason behind the missing data, we can better assess its impact whether it's causing bias or affecting the analysis and select the proper handling method such as imputation or removal.



Why Do We Need to Care About Handling Missing Data?

It is important to handle the missing values appropriately.

1. Many machine learning algorithms fail if the dataset contains missing values. However, algorithms like K-nearest and Naive Bayes support data with missing values.
2. You may end up building a biased machine learning model, leading to incorrect results if the missing values are not handled properly.
3. Missing data can lead to a lack of precision in the statistical analysis.



Methods for Managing Missing Data

- Detecting and managing missing data is important for data analysis.
- Let's see some useful functions for detecting, removing and replacing null values in Pandas DataFrame.

Functions	Descriptions
<u>.isnull()</u>	Detect missing values
<u>.notnull()</u>	Detect non-missing values
<u>.info()</u>	Summary with missing counts
<u>.isna()</u>	Same as isnull().
<u>dropna()</u>	Remove missing rows/columns
<u>fillna()</u>	Fill missing values

Handling Missing values

- Analyze each column with missing values carefully to understand the reasons behind the missing of those values, as this information is crucial to choose the strategy for handling the missing values.
- There are 2 primary ways of handling missing values:
 - Deleting the Missing values
 - Imputing the Missing Values



Deleting Missing Values

- **Deleting Missing Values:** This involves removing rows or columns with missing values.
- This is a straightforward method, but it can be problematic if a significant portion of your data is missing.
- Discarding too much data can affect the reliability of your conclusions.



Handling Missing values

- ✓ **Imputation of missing values:** This replaces missing values with estimates. Most machine learning algorithms require that their inputs have no missing values, and will not work if this requirement is violated. Algorithms that attempt to fill in (or impute) missing values are referred to as **imputation algorithms**.
 - There are various imputation techniques, each with its strengths and weaknesses.
 - ✓ • Imputing the Missing Values for numerical variables using pandas
 - ✓ • Imputing the Missing Values for Categorical Features variables using pandas
 - ✓ • Imputing the Missing Values using Sklearn Library
 - ✓ – Univariate imputation: SimpleImputer
 - ✓ – Multivariate imputation: KNNImputer



Imputing Missing values

1. Mean/Median/Mode Imputation:

- Replace missing entries with the
 - average (mean),
 - middle value (median), or
 - most frequent value (mode) of the corresponding column.
- This is a quick and easy approach, but it can introduce bias if the missing data is not randomly distributed.

2. K-Nearest Neighbors (KNN Imputation):

- This method finds the closest data points (neighbors) based on available features and uses their values to estimate the missing value.
- KNN is useful when you have a lot of data and the missing values are scattered.



Deleting Missing Values

- There are 2 ways one can delete the missing data values:
 - Deleting the entire row (listwise deletion)
 - Deleting the entire column

Checking for Missing Values in Python

- The first step in handling missing values is to carefully look at the complete data and find all the missing values.
- `info()`: Print a concise summary of a DataFrame. This method prints ~~information~~ about a DataFrame including the index dtype and columns, non-NA values and memory usage.



```
In [ ]:  ▶ import pandas as pd
          train_df = pd.read_csv("train_LoanPrediction.csv")
```

```
In [3]:  ▶ train_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 614 entries, 0 to 613
```

```
Data columns (total 13 columns):
```

#	Column	Non-Null Count	Dtype
0	Loan_ID	614 non-null	object
1	Gender	601 non-null	object
2	Married	611 non-null	object
3	Dependents	599 non-null	object
4	Education	614 non-null	object
5	Self_Employed	582 non-null	object
6	ApplicantIncome	614 non-null	int64
7	CoapplicantIncome	614 non-null	float64
8	LoanAmount	592 non-null	float64
9	Loan_Amount_Term	600 non-null	float64
10	Credit_History	564 non-null	float64
11	Property_Area	614 non-null	object
12	Loan_Status	614 non-null	object

```
dtypes: float64(4), int64(1), object(8)
```

```
memory usage: 62.5+ KB
```

O/P :

Checking for Missing Values in Python

- The following code shows total number of missing values in each column.

```
#Find the missing values from each column  
print(train_df.isnull().sum())
```

```
Loan_ID          0  
Gender           13  
Married          3  
Dependents       15  
Education        0  
Self_Employed    32  
ApplicantIncome  0  
CoapplicantIncome 0  
LoanAmount       22  
Loan_Amount_Term 14  
Credit_History  50  
Property_Area    0  
Loan_Status      0  
dtype: int64
```

- From the above output, we can see that there are 6 columns – Gender, Married, Dependents, Self_Employed, LoanAmount, Loan_Amount_Term, and Credit_History having missing values.



Checking for Missing Values in Python

- It also shows the total number of missing values in the entire data set.

```
In [18]: #Find the total number of missing values from the entire dataset  
train_df.isnull().sum().sum()
```

Out[18]: 149



Deleting the entire row (listwise deletion or Complete case analysis)

- Deleting the entire row (also known as listwise deletion or complete case analysis) is a simple method for handling missing values but is generally **only recommended when the amount of missing data is very small** (e.g., less than 5%) and the data is missing completely at random (MCAR).
- In most other scenarios, this approach is discouraged due to significant drawbacks.



Disadvantages of Deleting Entire Rows

- **Significant Data Loss:** The primary disadvantage is the loss of potentially valuable information. Even if a row has only one missing value in one column, the valid data in the other columns of that row are also discarded.
- **Reduced Sample Size and Statistical Power:** Removing many rows can drastically reduce the size of your dataset, which compromises the statistical power of your analysis and can lead to less precise parameter estimations.
- **Introduction of Bias:** If the data is not missing completely at random (MCAR), deleting rows can introduce bias. For example, if people with high incomes are less likely to report their earnings (Missing Not at Random - MNAR), removing those rows would skew your analysis and misrepresent the true population.
- **Poor Model Performance:** A model trained on a reduced or biased dataset may perform poorly on real-world, complete data.



Disadvantages of Deleting Entire Rows

- Instead of deleting rows, imputation is generally a more robust approach, as it retains all observations in the dataset by replacing missing values with estimated ones.



Deleting the entire row (listwise deletion)

- If a row has many missing values, you can drop the entire row. If every row has some (column) value missing, you might end up deleting the whole data. The code to drop the entire row is as follows:

```
df = train_df.dropna(axis=0)  
print(df.isnull().sum())  
df.info()
```

```
Loan_ID      0  
Gender       0  
Married      0  
Dependents   0  
Education    0  
Self_Employed 0  
ApplicantIncome 0  
CoapplicantIncome 0  
LoanAmount   0  
Loan_Amount_Term 0  
Credit_History 0  
Property_Area 0  
Loan_Status  0  
dtype: int64
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Int64Index: 480 entries, 1 to 613
```

```
Data columns (total 13 columns):
```

#	Column	Non-Null Count	Dtype
0	Loan_ID	480 non-null	object
1	Gender	480 non-null	object
2	Married	480 non-null	object
3	Dependents	480 non-null	object
4	Education	480 non-null	object
5	Self_Employed	480 non-null	object
6	ApplicantIncome	480 non-null	int64
7	CoapplicantIncome	480 non-null	float64
8	LoanAmount	480 non-null	float64
9	Loan_Amount_Term	480 non-null	float64
10	Credit_History	480 non-null	float64
11	Property_Area	480 non-null	object
12	Loan_Status	480 non-null	object

```
dtypes: float64(4), int64(1), object(8)
```

```
memory usage: 52.5+ KB
```



D Y PATIL
UNIVERSITY
NAVI MUMBAI

Dropna(how="all")

```
df = train_df.dropna(axis=0, how="all")  
print(df.isnull().sum())  
train_df.isnull().sum()
```

```
Loan_ID      0  
Gender       13  
Married      3  
Dependents   15  
Education    0  
Self_Employed 32  
ApplicantIncome 0  
CoapplicantIncome 0  
LoanAmount   22  
Loan_Amount_Term 14  
Credit_History 50  
Property_Area 0  
Loan_Status  0  
dtype: int64
```

```
Loan_ID      0  
Gender       13  
Married      3  
Dependents   15  
Education    0  
Self_Employed 32  
ApplicantIncome 0  
CoapplicantIncome 0  
LoanAmount   22  
Loan_Amount_Term 14  
Credit_History 50  
Property_Area 0  
Loan_Status  0
```



Deleting the entire column

- If a certain column has many missing values, then you can choose to drop the entire column. The code to drop the entire column is as follows:

```
df1 = train_df.dropna(axis=1)  
df1.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 614 entries, 0 to 613  
Data columns (total 6 columns):  
#   Column                Non-Null Count  Dtype  
---  ---  
0   Loan_ID               614 non-null   object  
1   Education             614 non-null   object  
2   ApplicantIncome       614 non-null   int64  
3   CoapplicantIncome     614 non-null   float64  
4   Property_Area         614 non-null   object  
5   Loan_Status           614 non-null   object  
dtypes: float64(1), int64(1), object(4)  
memory usage: 28.9+ KB
```

```
df = train_df.drop(['Dependents'],axis=1)  
df.isnull().sum()
```

```
] : Loan_ID           0  
    Gender           13  
    Married           3  
    Education         0  
    Self_Employed     32  
    ApplicantIncome    0  
    CoapplicantIncome  0  
    LoanAmount        22  
    Loan_Amount_Term   14  
    Credit_History     50  
    Property_Area      0  
    Loan_Status        0  
    dtype: int64
```



Imputing the Missing Value

- There are many imputation methods for replacing the missing values.
- You can use different python libraries such as Pandas, and Sci-kit Learn to do this.
- Let's go through some of the ways of replacing the missing values.
 - Replacing with an arbitrary value



Imputing the Missing Value: Replacing with an arbitrary value

- If you can make an educated guess about the missing value, then you can replace it with some arbitrary value using the following code.
- E.g., in the following code, we are replacing the missing values of the 'Dependents' column with 99.

```
#Replace the missing value with '99' using 'fillna' method  
train_df['Dependents'] = train_df['Dependents'].fillna(99)
```



- After filling missing values, check:

```
print(train_df.isnull().sum())
```

```
Loan_ID          0
Gender           13
Married          3
Dependents        0
Education         0
Self_Employed    32
ApplicantIncome   0
CoapplicantIncome 0
LoanAmount       22
Loan_Amount_Term  14
Credit_History   50
Property_Area     0
Loan_Status       0
dtype: int64
```

```
train_df[ 'Dependents' ].value_counts()
```

```
0      345
1      102
2      101
3+       51
99       15
Name: Dependents, dtype: int64
```



Imputing missing values of numeric column: Replacing with the mean

- This is the most common method of imputing missing values of numeric columns.
 - If there are outliers, then the mean will not be appropriate. In such cases, use median or treat outliers.
 - df.describe() method can be used to generate descriptive statistics about data.
 - You can use the 'fillna' method for imputing the columns 'LoanAmount' with the mean of the respective column values.



Data_frame.describe()

- The describe() method in pandas is a powerful function used to generate descriptive statistics that summarize the central tendency, dispersion, and shape of a dataset's distribution.
- Statistics Provided
 - The output of describe() varies depending on the data type of the column being analyzed.
 - For numeric data (e.g., integers and floats), the output includes:
 - count: Number of non-null observations.
 - mean: The average value.
 - std: Standard deviation, indicating how spread out the values are.
 - min and max: The minimum and maximum values.
 - 25%, 50%, 75%: The three quartiles (percentiles).



describe() in pandas

```
| train_df.describe()
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
count	614.000000	614.000000	592.000000	600.000000	564.000000
mean	5403.459283	1621.245798	146.412162	342.000000	0.842199
std	6109.041673	2926.248369	85.587325	65.12041	0.364878
min	150.000000	0.000000	9.000000	12.000000	0.000000
25%	2877.500000	0.000000	100.000000	360.000000	1.000000
50%	3812.500000	1188.500000	128.000000	360.000000	1.000000
75%	5795.000000	2297.250000	168.000000	360.000000	1.000000
max	81000.000000	41667.000000	700.000000	480.000000	1.000000



Replacing missing values with mean

```
#Replace the missing values for numerical columns with mean
```

```
train_df['LoanAmount'] = train_df['LoanAmount'].fillna(train_df['LoanAmount'].mean())
```

```
train_df.isnull().sum()
```

```
Loan_ID          0
Gender           13
Married          3
Dependents       0
Education        0
Self_Employed   32
ApplicantIncome  0
CoapplicantIncome 0
LoanAmount       0
Loan_Amount_Term 14
Credit_History  50
Property_Area    0
Loan_Status      0
dtype: int64
```



Replacing with the median

- The median is the middlemost value.
- It's better to use the median value for imputation in the case of outliers.
- You can use the 'fillna' method for imputing the column 'Loan_Amount_Term' with the median value.

```
[23]: In train_df['Loan_Amount_Term'] = train_df['Loan_Amount_Term'].fillna(train_df['Loan_Amount_Term'].median())  
      train_df.isnull().sum()
```

```
Out[23]: Loan_ID      0  
         Gender      0  
         Married     0  
         Dependents  0  
         Education   0  
         Self_Employed 0  
         ApplicantIncome 0  
         CoapplicantIncome 0  
         LoanAmount    0  
         Loan_Amount_Term 0  
         Credit_History 0  
         Property_Area  0  
         Loan_Status    0  
         dtype: int64
```

Handling categorical missing values: value_counts() in pandas

- **value_counts():**
 - Return a Series containing counts of unique values.
 - The resulting object will be in descending order so that the first element is the most frequently-occurring element. Excludes NA values by default.

```
In [16]: ▶ train_df['Gender'].value_counts()
```

```
Out[16]: Male      489  
        Female    112  
        Name: Gender, dtype: int64
```

```
In [13]: ▶ train_df['Gender'].mode()
```

```
Out[13]: 0      Male  
        dtype: object
```



Handling categorical missing values: mode()

- **mode()**

- The mode() function in pandas is used to find the most frequently occurring value(s) (the mode) in a Series or DataFrame.
- It is a versatile method that can be applied to both numeric and non-numeric data.

- **Key Characteristics**

- Returns a DataFrame (or Series for a Series object): The mode() function always returns a DataFrame (or a Series when applied to a Series), even if only one mode is found. This is because a dataset can have multiple modes (multiple values with the same highest frequency).
- Multiple Modes: If there are multiple modes, each is returned in its own row in the resulting DataFrame.



Imputing Categorical Variables: Replacing with the mode

- Mode is the most frequently occurring value.
- It is used in the case of categorical features.
- You can use the 'fillna' method for imputing the categorical columns 'Gender,' 'Married,' and 'Self_Employed.'

#Replace the missing values for categorical columns with mode

```
train_df['Gender'] = train_df['Gender'].fillna(train_df['Gender'].mode()[0])  
train_df['Married'] = train_df['Married'].fillna(train_df['Married'].mode()[0])  
train_df['Self_Employed'] = train_df['Self_Employed'].fillna(train_df['Self_Employed'].mode()[0])  
train_df['Credit_History'] = train_df['Credit_History'].fillna(train_df['Credit_History'].mode()[0])  
train_df.isnull().sum()
```

Loan_ID	0
Gender	0
Married	0
Dependents	0
Education	0
Self_Employed	0
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	0
Loan_Amount_Term	0
Credit_History	0
Property_Area	0
Loan_Status	0
dtypes: int64	



Imputation of missing values using scikit-learn

- **Univariate vs. Multivariate Imputation:**

- One type of imputation algorithm is univariate, which imputes values in the i -th feature dimension using only non-missing values in that feature dimension (e.g. **SimpleImputer**).
- By contrast, multivariate imputation algorithms use the entire set of available feature dimensions to estimate the missing values (e.g. **KNNImputer**).



Univariate feature imputation using SimpleImputer

- **SimpleImputer** is a scikit-learn class which provides basic strategies for imputing the missing data in the predictive model dataset.
 - It replaces the NaN values with a specified placeholder.
 - constant value, or
 - using the statistics of each column in which the missing values are located.
 - mean,
 - median or
 - most frequent
- This class also allows for different missing values encodings.



Univariate feature imputation using SimpleImputer

It is implemented by the use of the **SimpleImputer()** method which takes the following arguments :

- **Parameters of the SimpleImputer class:**

- **missing_values:** indicates the missing values in the dataset. By default np.nan is set in the missing_values, which means all the values containing np.nan will be considered as missing values.
- **strategy:** it is the method by using which we want to fill in the missing values. the value of the strategy could be “mean”, “median”, “most_frequent”, or “constant”.
- **fill_value:** It is a parameter only used when the strategy is set to be constant. When the strategy is constant, the Nan values will be replaced by the value passes in fill_value.

- **Remember: The mean or median is taken along the column of the matrix**



Univariate feature imputation using SimpleImputer: imputation strategy

- **imputation strategy:**
 - If “**mean**”, then replace missing values using the mean along each column. Can only be used with numeric data.
 - If “**median**”, then replace missing values using the median along each column. Can only be used with numeric data.
 - If “**most_frequent**”, then replace missing using the most frequent value along each column. Can be used with strings or numeric data. If there is more than one such value, only the smallest is returned.
 - If “**constant**”, then replace missing values with fill_value. Can be used with strings or numeric data.



Example of imputation using SimpleImputer

```
import numpy as np

# Importing the SimpleImputer class
from sklearn.impute import SimpleImputer

# Imputer object using the mean strategy and
# missing_values type for imputation
imputer = SimpleImputer(missing_values = np.nan,
                        strategy = 'mean')

data = [[12, np.nan, 34],
        [10, 32, np.nan],
        [np.nan, 11, 20]]

# Fitting the data to the imputer object
imputer = imputer.fit(data)

# Imputing the data
data = imputer.transform(data)

print("Imputed Data : \n", data)
```

```
Imputed Data :
[[12.  21.5 34. ]
 [10.  32.  27. ]
 [11.  11.  20. ]]
```

Can also use `data = imputer.fit_transform(data)`



SimpleImputer Strategy: mean

Mean Imputation: Mean imputation replaces missing values in numerical columns with the mean of the non-missing values for that column.

```
import pandas as pd
from sklearn.impute import SimpleImputer

# Sample Data
data = {'Age': [25, 27, 29, None, 32, 35, None, 40],
        'Salary': [50000, 54000, 58000, 60000, None, 65000, None, 70000]}
df = pd.DataFrame(data)
print(df)
# Imputer for Mean Strategy
mean_imputer = SimpleImputer(strategy='mean')

# Apply imputation
df['Age'] = mean_imputer.fit_transform(df[['Age']])
df['Salary'] = mean_imputer.fit_transform(df[['Salary']])
print(df)
```

	Age	Salary
0	25.0	50000.0
1	27.0	54000.0
2	29.0	58000.0
3	NaN	60000.0
4	32.0	NaN
5	35.0	65000.0
6	NaN	NaN
7	40.0	70000.0

	Age	Salary
0	25.000000	50000.0
1	27.000000	54000.0
2	29.000000	58000.0
3	31.333333	60000.0
4	32.000000	59500.0
5	35.000000	65000.0
6	31.333333	59500.0
7	40.000000	70000.0

SimpleImputer Strategy: median

- **Median Imputation:** Median imputation replaces missing values in numerical columns with the median of the non-missing values. This is helpful when the data is skewed.

```
# SimpleImputer strategy: median
import pandas as pd
from sklearn.impute import SimpleImputer

# Sample Data
data = {'Age': [25, 27, 29, None, 32, 35, None, 40],
        'Salary': [50000, 54000, 58000, 60000, None, 65000, None, 70000]}
df = pd.DataFrame(data)
print(df)

# Imputer for Median Strategy
median_imputer = SimpleImputer(strategy='median')

# Apply imputation
df['Age'] = median_imputer.fit_transform(df[['Age']])
df['Salary'] = median_imputer.fit_transform(df[['Salary']])
print(df)
```

	Age	Salary
0	25.0	50000.0
1	27.0	54000.0
2	29.0	58000.0
3	NaN	60000.0
4	32.0	NaN
5	35.0	65000.0
6	NaN	NaN
7	40.0	70000.0

	Age	Salary
0	25.0	50000.0
1	27.0	54000.0
2	29.0	58000.0
3	30.5	60000.0
4	32.0	59000.0
5	35.0	65000.0
6	30.5	59000.0
7	40.0	70000.0



SimpleImputer Strategy: most_frequent

- **Mode Imputation:** Mode imputation is used for categorical data where missing values are filled with the most frequent value (mode).
- In case of ties it selects the smallest value in sorted order.

```
| import pandas as pd
| from sklearn.impute import SimpleImputer
| # Sample Data
| data = {'Gender': ['Male', 'Female', None, 'Male', None, 'Female', 'Female', 'Male']}
| df = pd.DataFrame(data)
|
| # Imputer for Mode Strategy (Most Frequent)
| mode_imputer = SimpleImputer(missing_values=None, strategy='most_frequent')
| #print(df)
| print()
| # Apply imputation
| df['Gender'] = mode_imputer.fit_transform(df[['Gender']])
| print(df)
```

```
Gender
0    Male
1  Female
2  Female
3    Male
4  Female
5  Female
6  Female
7    Male
```

```

import pandas as pd
from sklearn.impute import SimpleImputer
# Sample Data
data = {'Gender': ['Male', 'Female', None, 'Male', None, 'Female', 'Female', 'Male', 'Male']}
df = pd.DataFrame(data)

# Imputer for Mode Strategy (Most Frequent)
mode_imputer = SimpleImputer(missing_values=None, strategy='most_frequent')
#print(df)
print()
# Apply imputation
df['Gender'] = mode_imputer.fit_transform(df[['Gender']])
print(df)

```

	Gender
0	Male
1	Female
2	Male
3	Male
4	Male
5	Female
6	Female
7	Male
8	Male



Multivariate Approach

- In a multivariate approach, more than one feature is taken into consideration.
- In multivariate data imputation, we utilize observations from other variables in the dataset to estimate the values of missing observations.
- KNNImputer class can impute missing values considering the multivariate approach.



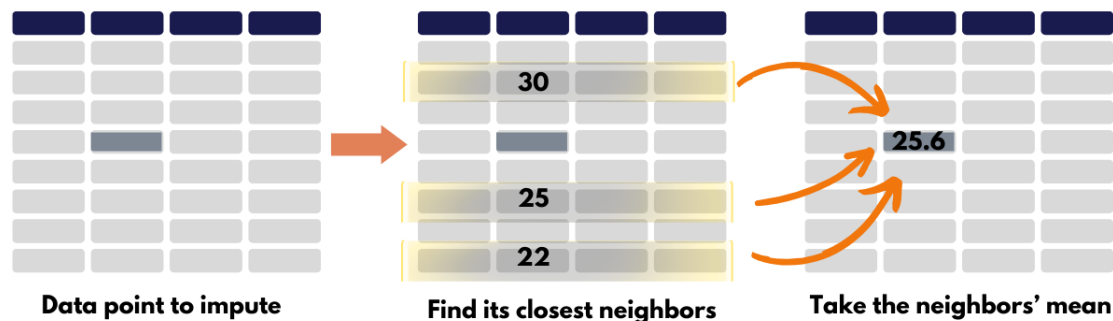
KNNImputer

- **KNNImputer** is a `_scikit-learn` class used to fill out or predict the missing values in a dataset.
- The KNN Imputer is a machine learning–based method for filling missing values in datasets. Instead of using a single statistic (like mean or median), it estimates missing values using the values of the k most similar data points (neighbors).
 - **Multivariate approach:** Considers multiple features simultaneously.
 - **Data-driven:** Uses patterns in the dataset, not external assumptions.
 - **Robust:** Better preserves relationships between variables compared to univariate methods.



How Does KNNImputer Work?

1. The KNNImputer works by finding the k-nearest neighbors (based on a specified distance metric) for the data points with missing values.
2. It then imputes the missing values using the mean or median (depending on the specified strategy) of the neighboring data points.
3. The key advantage of this approach is that it preserves the relationships between features, which can lead to better model performance.



How Does KNNImputer Work?

4. For example, consider a dataset with a missing value in a column representing a student's math score.
5. Instead of simply filling this missing value with the overall mean or median of the math scores, KNNImputer finds the k-nearest students (based on other features like scores in physics, chemistry, etc.) and imputes the missing value using the mean or median of these neighbors' math scores.



Example data: KNNImputer

Row	Maths	Chemistry	Physics	Biology
0	80	60	NaN	78
1	90	65	57	83
2	NaN	56	80	67
3	95	NaN	78	NaN



Final Imputed Table

Row	Maths	Chem	Physics	Bio
0	80	60	68.5	78
1	90	65	57	83
2	87.5	56	80	67
3	95	58	78	72.5



KNNImputer()

- class `sklearn.impute.KNNImputer(*, missing_values=nan, n_neighbors=5, weights='uniform', metric='nan_euclidean')`
 - **missing_values**
 - The placeholder for missing values. All occurrences of missing_values will be imputed. **default=np.nan**
 - **n_neighbors**: *number of data points to include closer to the missing value. default= 5*
 - **metric**: *the distance metric to be used for searching. values - {nan_euclidean, callable}. default = **nan_euclidean***
 - **weights**: *to determine on what basis should the neighboring values be treated values : {uniform, distance, callable}. default= **uniform**.*
- *Note: A callable in Python is a general term for any object that can be called like a function*

In this imputation we take help of other columns using NaN-Eucliden-Distance

$$distance = \sqrt{weight * (x_1 - x_2)^2 + (y_1 - y_2)^2}$$

where,

$$weight = \frac{Totalno.ofcoordinate}{Totalno.ofavailablecoordinate}$$

KNNimputer(): weights

- Possible values of weights :
 - ‘uniform’ : uniform weights.
 - All points in each neighborhood are weighted equally.
 - ‘distance’ : weight points by the inverse of their distance.
 - in this case, closer neighbors of a query point will have a greater influence than neighbors which are further away.
 - callable : a user-defined function which accepts an array of distances, and returns an array of the same shape containing the weights.



Ex: KNNImputer

```
# Example of KNNImputer
import numpy as np
import pandas as pd

# import the KNNImputer class
from sklearn.impute import KNNImputer

# create dataset for marks of a student
dict = {'Maths': [80, 90, np.nan, 95],
        'Chemistry': [60, 65, 56, np.nan],
        'Physics': [np.nan, 57, 80, 78],
        'Biology': [78, 83, 67, np.nan]}

# creating a data frame from the list
Before_imputation = pd.DataFrame(dict)
# print dataset before imputation
print("Data Before performing imputation\n", Before_imputation)

# create an object for KNNImputer
imputer = KNNImputer(n_neighbors=2)
After_imputation = imputer.fit_transform(Before_imputation)
# print dataset after performing the operation
print("\n\nAfter performing imputation\n", After_imputation)
```

Data Before performing imputation

	Maths	Chemistry	Physics	Biology
0	80.0	60.0	NaN	78.0
1	90.0	65.0	57.0	83.0
2	NaN	56.0	80.0	67.0
3	95.0	NaN	78.0	NaN

After performing imputation

```
[[80.  60.  68.5 78. ]
 [90.  65.  57.  83. ]
 [87.5 56.  80.  67. ]
 [95.  58.  78.  72.5]]
```

Advantages of Using KNNImputer

- **Multivariate approach:** Considers correlations between features. By using the k-nearest neighbors, this method preserves the relationships between features, which can improve model performance..
- **Customizable:** The ability to customize the number of neighbors, distance metric, and weighting scheme makes KNNImputer highly versatile and adaptable to different types of data.
- **Preserves distribution:** Maintains dataset integrity better than mean/median filling.



Limitations of KNNImputer

- **Computationally Intensive:** Finding the k-nearest neighbors for each missing value can be computationally expensive, especially for large datasets with many missing values.
- **Choice of k:** Small k may overfit, large k may oversmooth important details.
- **Memory usage:** Need to store training dataset in production to Handle upcoming missing data
- **KNN Imputer is sensitive to scale:** Because it is a distance-based algorithm (typically using Euclidean distance by default), features with larger value ranges or magnitudes can dominate the distance calculations, leading to biased and inaccurate imputation results. Therefore, Real-world workflow should include scaling



Limitations of KNNImputer

- **Not ideal for categorical data:** KNN imputation replaces missing values as the weighted average of the closest neighbors to the observations with nan values. This means that this imputation method can only be applied to impute numerical variables.
- In addition, the nearest neighbor algorithm computes distances between observations, hence, it is only suitable for numeric data. If your dataset contains variables with other data types, such as datetime or categorical variables, you'll need to encode them first, or try a different missing value imputation technique.



Summary: KNNImputer

- KNNImputer in Scikit-Learn is a powerful tool for handling missing data, offering a more sophisticated alternative to traditional imputation methods.
- By leveraging the relationships between features, it provides more accurate imputations that can lead to better model performance.
- However, it is essential to be mindful of its computational demands and sensitivity to outliers.
- When used appropriately, KNNImputer can significantly enhance your_data preprocessing pipeline, leading to more robust and reliable machine-learning models.



Feature 	<u>SimpleImputer</u>	<u>KNNImputer</u>
Approach	Univariate: Computes the imputation value using only non-missing values in the specific feature (column) where the value is missing.	Multivariate: Estimates missing values by considering the values of all other features in a given data point to find its neighbors.
Strategy	Replaces missing values with a basic statistic like the mean, median, or most frequent value.	Replaces missing values with the mean value (uniformly or distance-weighted) of its <code>k</code> nearest neighbors.
Complexity	Simple and fast to execute.	More computationally intensive and slower, especially for large datasets.
Data Types	Works well with both numerical and categorical data (using 'most frequent' or 'constant' strategies).	Primarily designed for numerical features; requires separate handling for categorical data if not encoded numerically.
Accuracy	Less accurate as it doesn't consider relationships between different features.	Generally more accurate because it leverages feature correlations and patterns in the data.
Scaling	Does not require feature scaling to work correctly.	Requires data to be scaled (e.g., using <code>MinMaxScaler</code> or <code>StandardScaler</code>) as it is distance-based.
Use Case	Best for small amounts of missing data, or when missingness is not related to other features.	Recommended when there are many missing values, and the data has strong correlations between features.

Encoding categorical data

- Categorical variables are those whose values are selected from a group of categories or labels.
- Their values can be strings or numbers.
- They are finite in number.
- *Color* is a categorical feature that can take values such as *red*, *yellow* or *orange*, among others.
- Similarly, *size* is another categorical feature, with the values of *small*, *medium* and *large*.
- There are two types of categorical data -
 - *Ordinal Data*
 - *Nominal Data*



Types of Categorical Data

- **Nominal features** are categorical features whose categories don't have a defined ranking or inherent order.
 - *Color* is an example of a nominal feature because each color is an independent entity, without a logical ordering or ranking.
- **Ordinal features** are categorical features whose categories show an inherent order or ranking.
 - *Size* is an example of an ordinal feature, as the different sizes can be ordered from smallest to largest.
- Understanding if categorical feature are *nominal* or *ordinal* can help us choose the most appropriate encoding method to transform their values into numbers.



What is Data Encoding?

- **Data Encoding** is an important pre-processing step in Machine Learning.
- *It refers to the process of converting categorical or textual data into numerical format, so that it can be used as input for algorithms to process.*
- The reason for encoding is that most machine learning algorithms work with numbers and not with text or categorical variables.



Importance of categorical encoding

- Most machine learning algorithms work only with numerical data, so categorical variables (such as text labels) must be transformed into numerical values.
- This allows the model to identify patterns in the data and make predictions based on those patterns.
- Encoding also helps to prevent bias in the model by ensuring that all features are equally weighted.
- The choice of encoding method can have a significant impact on model performance, so it is important to choose an appropriate encoding technique based on the nature of the data and the specific requirements of the model.



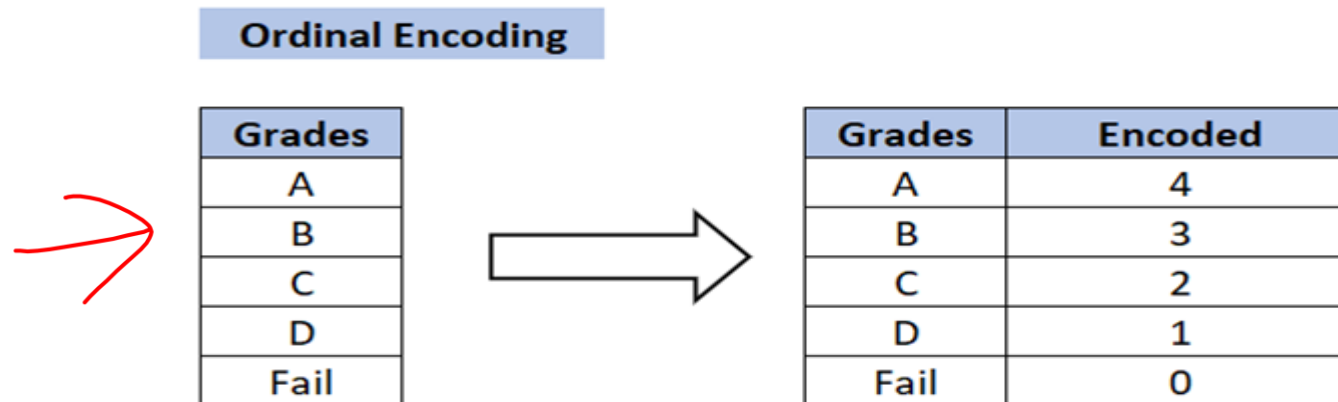
Steps for Encoding Categorical Variables in Machine Learning

1. **Identify Categorical Features:** First, look at your data and find the features that contain non-numerical values like text labels or categories (e.g., color, size, customer type). These are the features that need encoding.
2. **Choose an Encoding Technique:** There are different encoding methods, each with its pros and cons.
 - **Ordinal Encoding**
 - **One-Hot Encoding**
 - **Label Encoding**
3. **Apply the Encoding:** Once you've chosen your technique, it's time to apply it to your data. Many machine learning libraries have built-in functions for encoding.
4. **Test and Refine (Optional):** In some cases, you might want to try different encoding techniques and see how they affect your machine learning model's performance. This can help you find the best approach for your specific data.



Ordinal Encoding

- Ordinal Encoding is used when the **categories** in a variable have a **Natural Ordering**.
- In this method, the **categories are assigned a numerical value** based on their order, such as 0, 1, 2, etc.
- For example, if a variable has categories '**Low**', '**Medium**' and '**High**', they can be assigned the values **0, 1, 2**, respectively.



Applying Ordinal Encoding using sklearn.preprocessing.OrdinalEncoder

```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder

# 1. Create a sample DataFrame
data = {
    'Education': ['High School', 'Bachelor', 'Master', 'PhD', 'Bachelor'],
    'Satisfaction': ['Poor', 'Good', 'Excellent', 'Average', 'Good']
}
df = pd.DataFrame(data)

# 2. Define the explicit order for each feature
education_order = ['High School', 'Bachelor', 'Master', 'PhD']
satisfaction_order = ['Poor', 'Average', 'Good', 'Excellent']

# The 'categories' parameter expects a list of lists, one for each column in order
encoder = OrdinalEncoder(categories=[education_order, satisfaction_order])

# 3. Fit and transform the data
# We select the columns to encode as a list of lists (or a DataFrame)
df_encoded = encoder.fit_transform(df[['Education', 'Satisfaction']])

# Convert the result back to a DataFrame for better readability
df_encoded = pd.DataFrame(df_encoded, columns=['Education_Encoded', 'Satisfaction_Encoded'])

# Display the result
print("Original DataFrame:")
print(df)
print("\nEncoded DataFrame:")
print(df_encoded)
```

Original DataFrame:

	Education	Satisfaction
0	High School	Poor
1	Bachelor	Good
2	Master	Excellent
3	PhD	Average
4	Bachelor	Good

Encoded DataFrame:

	Education_Encoded	Satisfaction_Encoded
0	0.0	0.0
1	1.0	2.0
2	2.0	3.0
3	3.0	1.0
4	1.0	2.0



Label Encoder

- ***class* sklearn.preprocessing.LabelEncoder**
 - Encode target labels with value between 0 and `n_classes-1`.
 - This transformer should be used to encode target values, *i.e.* `y`, and not the input `X`



Example code: Label Encoding

```
# Label Encoding Demo – Loan Approval Target
import pandas as pd
from sklearn.preprocessing import LabelEncoder
df = pd.DataFrame({
    'ApplicantIncome': [5000, 3000, 4000, 6500, 2000],
    'Education': ['Graduate', 'Not Graduate', 'Graduate', 'Graduate', 'Not Graduate'],
    'Loan_Status': ['Approved', 'Rejected', 'Approved', 'Approved', 'Rejected']
})

print("Original Dataset:\n")
print(df)

# Apply Label Encoding to TARGET
le = LabelEncoder()
df['Loan_Status_Encoded'] = le.fit_transform(df['Loan_Status'])

print("\nEncoded Dataset:\n")
print(df)
```

Original Dataset:

	ApplicantIncome	Education	Loan_Status
0	5000	Graduate	Approved
1	3000	Not Graduate	Rejected
2	4000	Graduate	Approved
3	6500	Graduate	Approved
4	2000	Not Graduate	Rejected

Encoded Dataset:

	ApplicantIncome	Education	Loan_Status	Loan_Status_Encoded
0	5000	Graduate	Approved	0
1	3000	Not Graduate	Rejected	1
2	4000	Graduate	Approved	0
3	6500	Graduate	Approved	0
4	2000	Not Graduate	Rejected	1



Ordinal Encoding verses Label Encoding

Feature	Ordinal Encoding	Label Encoding
Data Type	Used for ordinal data (data with a natural rank, e.g., "low", "medium", "high").	Used for nominal data (data without an inherent order, e.g., "red", "blue", "green").
Purpose	Preserves the intrinsic order of categories, so the numerical representation reflects the hierarchy.	Assigns a unique number to each category in an arbitrary manner, usually alphabetically in tools like scikit-learn's LabelEncoder.
Typical Use Case	Encoding independent features (X) in a dataset where the order is important to the model.	Primarily used for encoding the target variable (y) in classification tasks.
Implementation	The specific order can be manually defined (e.g., small=0, medium=1, large=2).	Numbers are typically assigned in an ascending, alphabetical order unless a manual mapping is specified.



```

# Ordinal Encoding + Label Encoding + Train/Test Split
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder

# Sample dataset
df = pd.DataFrame({
    'Education': ['Not Graduate', 'Graduate', 'Postgraduate', 'Graduate',
                  'Not Graduate', 'Postgraduate', 'Graduate', 'Postgraduate'],
    'Income': [3000, 5000, 6500, 4500, 2000, 7200, 5200, 6100],
    'Loan_Status': ['Rejected', 'Approved', 'Approved', 'Approved',
                    'Rejected', 'Approved', 'Approved', 'Rejected']
})

# Split features and target
X = df[['Education', 'Income']]
y = df['Loan_Status']

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Ordinal Encoding (fit ONLY on train)
order = [['Not Graduate', 'Graduate', 'Postgraduate']]
ord_enc = OrdinalEncoder(categories=order)
X_train[['Education']] = ord_enc.fit_transform(X_train[['Education']])
X_test[['Education']] = ord_enc.transform(X_test[['Education']])

# Label Encoding Target
le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)
y_test_enc = le.transform(y_test)

# Display results
print("X_train:\n", X_train)
print("\nX_test:\n", X_test)
print("\ny_train encoded:\n", y_train_enc)
print("\ny_test encoded:\n", y_test_enc)

```

X_train:

	Education	Income
7	2.0	6100
2	2.0	6500
4	0.0	2000
3	1.0	4500
6	1.0	5200

X_test:

	Education	Income
1	1.0	5000
5	2.0	7200
0	0.0	3000

y_train encoded:
[1 0 1 0 0]

y_test encoded:
[0 0 1]



One Hot Encoding

- We use this categorical data encoding technique when the features are nominal(do not have any order).
- In one hot categorical encoding, for each level of a categorical feature, we create a new variable.
- Each category is mapped with a binary variable containing either 0 or 1.
 - Here, 0 represents the absence, and
 - 1 represents the presence of that category.
- These newly created binary features are known as **Dummy variables**.
- The number of dummy variables depends on the levels present in the categorical variable.
- .



One Hot Encoding

- Suppose we have a dataset with a category animal, having different animals like Dog, Cat, Sheep, Cow, Lion.
- After one hot encoding, in the second table, we have dummy variables each representing a category in the feature Animal.
- Now for each category that is present, we have 1 in the column of that category and 0 for the others.

Index	Animal	One-Hot code	Index	Dog	Cat	Sheep	Lion	Horse
0	Dog		0	1	0	0	0	0
1	Cat		1	0	1	0	0	0
2	Sheep		2	0	0	1	0	0
3	Horse		3	0	0	0	0	1
4	Lion		4	0	0	0	1	0



One Hot Encoding

```
# One-Hot Encoding
```

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder
```

```
# Sample dataset
```

```
df = pd.DataFrame({
    'Gender': ['Male', 'Female', 'Female', 'Male'],
    'Married': ['Yes', 'No', 'Yes', 'No']
})
```

```
print("Original Data:\n")
print(df)
```

```
# Create encoder.. in new versions sparse_output in new versions
encoder = OneHotEncoder(sparse=False)
```

```
# Fit and transform
```

```
encoded_array = encoder.fit_transform(df)
```

```
# Convert to DataFrame for readability get_feature_names_out in new versions
```

```
encoded_df = pd.DataFrame(
    encoded_array,
    columns=encoder.get_feature_names(df.columns)
)
```

```
print("\nOne-Hot Encoded Data:\n")
print(encoded_df)
```

Original Data:

	Gender	Married
0	Male	Yes
1	Female	No
2	Female	Yes
3	Male	No

One-Hot Encoded Data:

	Gender_Female	Gender_Male	Married_No	Married_Yes
0	0.0	1.0	0.0	1.0
1	1.0	0.0	1.0	0.0
2	1.0	0.0	0.0	1.0
3	0.0	1.0	1.0	0.0

sparse_output=False)



One Hot Encoding

- Size of one-hot vectors is equal to the number of unique values that a categorical column takes up and each such vector contains exactly one '1' in it.
- This ingests **multicollinearity** into our dataset.
 - From the encoded dataset, we can observe the following linear relationship
 - $\text{dummy_variable}_1 + \text{dummy_variable}_2 + \dots + \text{dummy_variable}_n = 1$
- The vectors that we use to encode the categorical columns are called '***Dummy Variables***'.
- We intended to solve the problem of using categorical variables, but got *trapped* by the problem of Multicollinearity. This is called the ***Dummy Variable Trap***.
- As mentioned earlier, this directly impacts the linear regression analysis because linear models (eg: linear regression, logistic regression, etc) assumes non-existence of multicollinearity in the dataset.



Multicollinearity

- Multicollinearity is a phenomenon in multiple regression analysis where two or more independent (predictor) variables are highly correlated, meaning they provide redundant information.
- This linear dependency makes it difficult to isolate the individual effect of each predictor on the dependent variable, leading to unstable, unreliable, and difficult-to-interpret regression coefficients.
- e.g., predicting house prices using both "square footage" and "number of rooms".

-



```

#drop='first'
# One-Hot Encoding

import pandas as pd
from sklearn.preprocessing import OneHotEncoder

# Sample dataset
df = pd.DataFrame({
    'Gender': ['Male', 'Female', 'Female', 'Male'],
    'Married': ['Yes', 'No', 'Yes', 'No']
})

print("Original Data:\n")
print(df)

# Create encoder.. in new versions sparse_output in new versions
encoder = OneHotEncoder(sparse=False, drop='first')

# Fit and transform
encoded_array = encoder.fit_transform(df)

# Convert to DataFrame for readability get_feature_names_out in new versions
encoded_df = pd.DataFrame(
    encoded_array,
    columns=encoder.get_feature_names(df.columns)
)

print("\nOne-Hot Encoded Data:\n")
print(encoded_df)

```

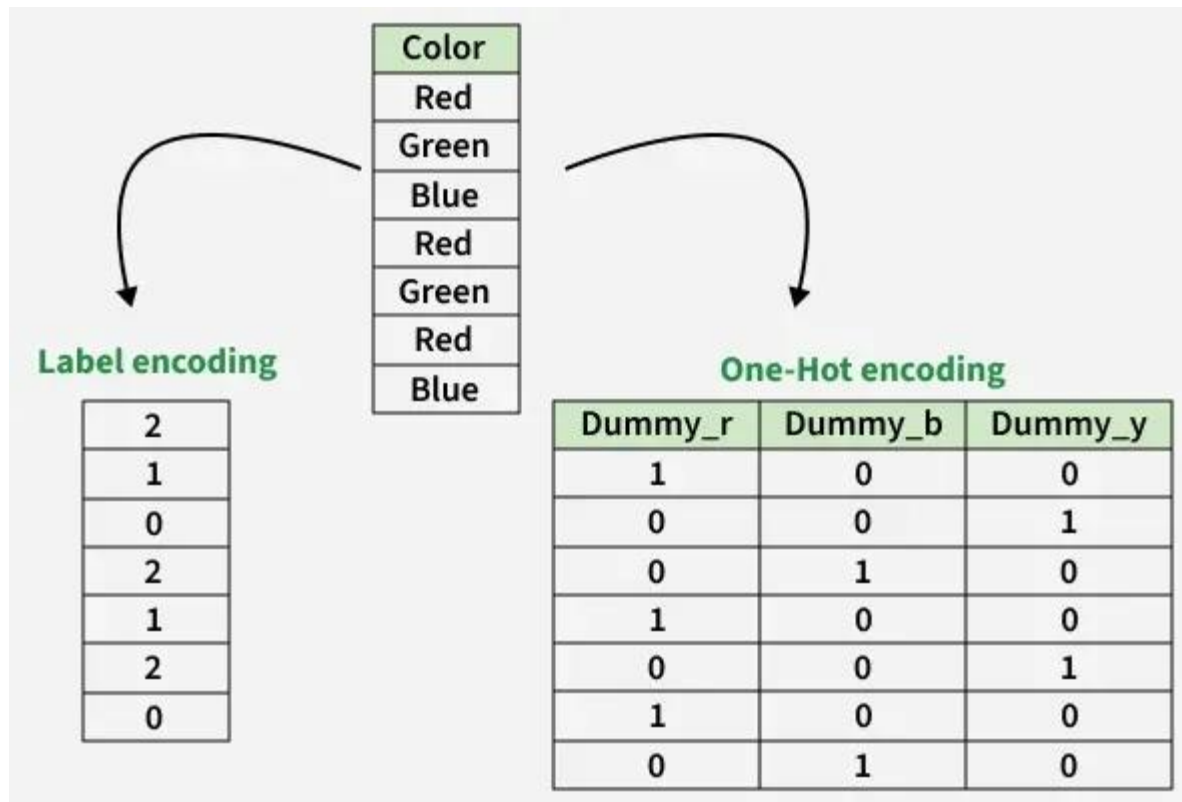
Original Data:

	Gender	Married
0	Male	Yes
1	Female	No
2	Female	Yes
3	Male	No

One-Hot Encoded Data:

	Gender_Male	Married_Yes
0	1.0	1.0
1	0.0	0.0
2	0.0	1.0
3	1.0	0.0

One Hot Encoding vs Label encoding



Feature scaling

- Feature scaling is a critical preprocessing step in machine learning that transforms the numerical features of a dataset into a common, uniform range.
- Without it, features with larger magnitudes (like income in thousands) can disproportionately influence a model compared to features with smaller ranges (like age in years), leading to biased or inaccurate results



Feature Scaling

- With few exceptions, Machine Learning algorithms don't perform well when the input numerical attributes have very different scales.
- **It ensures all features contribute equally, prevents large-magnitude features from dominating, and speeds up convergence for gradient-based algorithms.**
- **Feature Scaling is critical for algorithms using distance metrics (e.g., KNN, SVM, K-Means), linear models, gradient descent based algorithms (e.g., Neural Networks).**
- **When it is Not Needed: Tree-based models (e.g., Random Forest, Gradient Boosting) and Naive Bayes are generally invariant to feature scales**
- Note that scaling the target values is generally not required.



Key Reasons for Feature Scaling:

- **Preventing Feature Dominance:** Without scaling, features with larger ranges (e.g., income in thousands) overshadow smaller ones (e.g., age in years) in distance-based calculations.
- **Faster Convergence:** Gradient-based optimization algorithms converge much faster when features are on a similar scale, reducing oscillation.
- **Improved Model Performance:** Models like Support Vector Machines (SVM) and PCA are sensitive to data magnitude and perform better when data is normalized or standardized.
- **Numerical Stability:** It reduces risks of numerical overflow or underflow during complex calculations



Common Methods for Feature Scaling

- **Standardization (Z-score normalization):** Centers data with a mean of 0 and a standard deviation of 1, handling outliers better.
- **Normalization (Min-Max Scaling):** Scales data to a fixed range, usually.



StandardScaler

- The **Standardization** is a data preprocessing technique used in machine learning to **standardize** features by removing the mean and scaling the data to unit variance.
- This process, also known as **z-score normalization**.
- It transforms each feature to have a mean of 0 and a standard deviation of 1.
- Thus, StandardScaler() will **normalize the features** i.e. each column of X, *individually* so that each column/feature/variable will have $\mu = 0$ and $\sigma = 1$.

Standardization:

$$z = \frac{x - \mu}{\sigma}$$

with mean:

$$\mu = \frac{1}{N} \sum_{i=1}^N (x_i)$$

and standard deviation

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$



UNIVERSITY
NAVI MUMBAI

Standardarization

- **Advantages**

- It handles features with different units effectively.
- It reduces impact of outliers without completely removing them.

- **Disadvantages**

- It is sensitive to outliers and extreme values can skew mean and standard deviation which leads to poor scaling.
- It is not ideal for non-normal distributions.



Example1: StandardScaler

```
from sklearn.preprocessing import StandardScaler
import numpy as np

# Sample data with different scales
# Feature 1 (e.g., Age) has values from 0 to 11
# Feature 2 (e.g., Experience) has values from 2 to 10
data = np.array([[11, 2],
                 [3, 7],
                 [0, 10],
                 [11, 8]])

# 1. Create an instance of the StandardScaler
scaler = StandardScaler()

# 2. Fit the scaler to the data and transform the data
scaled_data = scaler.fit_transform(data)

# 3. Print the original and scaled data
print("Original Data:\n", data)
print("\nScaled Data (mean 0, std dev 1):\n", scaled_data)
```

Original Data:

```
[[11  2]
 [ 3  7]
 [ 0 10]
 [11  8]]
```

Scaled Data (mean 0, std dev 1):

```
[[ 0.97596444 -1.61155897]
 [-0.66776515  0.08481889]
 [-1.28416374  1.10264561]
 [ 0.97596444  0.42409446]]
```



Example Code 2: StandardScaling

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
df=pd.read_csv("Social_Network_Ads.csv")
df.head()
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0



```
df=df.iloc[:,2:]  
df
```

	Age	EstimatedSalary	Purchased
0	19	19000	0
1	35	20000	0
2	26	43000	0
3	27	57000	0
4	19	76000	0
...	...	...	...
395	46	41000	1
396	51	23000	1
397	50	20000	1
398	36	33000	0
399	49	36000	1

400 rows × 3 columns

```
#Splitting data into train set and test set
from sklearn.model_selection import train_test_split
X_train,X_test, y_train, y_test=train_test_split(df.drop("Purchased", axis=1), df["Purchased"], test_size=0.3, random_state=0)
```

```
X_train.shape, X_test.shape
```

```
: ((280, 2), (120, 2))
```



Standard Scaler

```
from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
X_train_scaled=scaler.fit_transform(X_train)
X_test_scaled=scaler.transform(X_test)
#
X_train_scaled=pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_test_scaled=pd.DataFrame(X_test_scaled, columns=X_test.columns)
X_train,X_train_scaled
```

```
10]: (   Age  EstimatedSalary
     92      26          15000
    223      60         102000
    234      38         112000
    232      40         107000
    377      42          53000
     ..      ...          ...
    323      48          30000
    192      29          43000
    117      36          52000
     47      27          54000
    172      26         118000

[280 rows x 2 columns],
      Age  EstimatedSalary
0  -1.163172    -1.584970
1   2.170181     0.930987
2   0.013305     1.220177
3   0.209385     1.075582
4   0.405465    -0.486047
..      ...          ...
275  0.993704    -1.151185
```



▶ `np.round(X_train.describe()), np.round(X_train_scaled.describe())`

```
: (
  count    280.0    280.0
  mean     38.0    69807.0
  std      10.0    34641.0
  min      18.0    15000.0
  25%      30.0    43000.0
  50%      37.0    70500.0
  75%      46.0    88000.0
  max      60.0   150000.0,
  count    280.0    280.0
  mean      0.0      0.0
  std       1.0      1.0
  min      -2.0     -2.0
  25%      -1.0     -1.0
  50%      -0.0      0.0
  75%       1.0      1.0
  max       2.0      2.0)
```

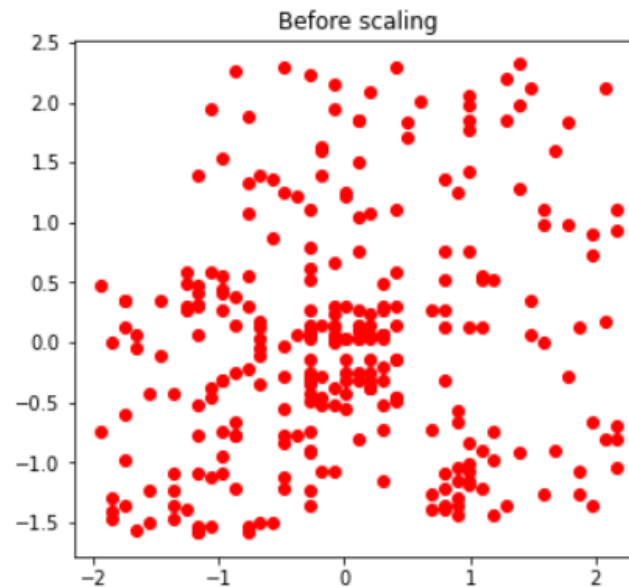
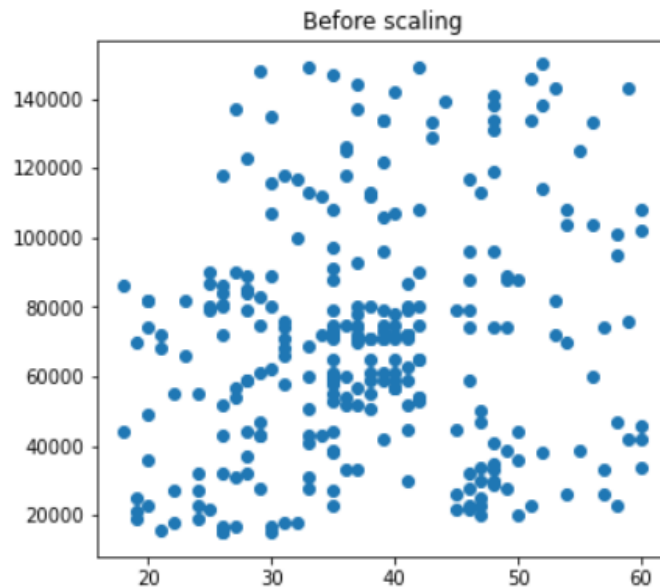


plt.scatter()

The plt.scatter() function in the matplotlib.pyplot module is used to create scatter plots, which visualize the relationship between two or more numerical variables by displaying data points as individual markers.

```
2]: In [12]: fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12, 5))
ax1.scatter(X_train['Age'], X_train['EstimatedSalary'])
ax1.set_title("Before scaling")
ax2.scatter(X_train_scaled['Age'], X_train_scaled['EstimatedSalary'], color='red')
ax2.set_title("Before scaling")
```

In [12]: Text(0.5, 1.0, 'Before scaling')



Normalization (Min-Max Scaling)

- **Purpose:** Rescales data to a fixed range (often [0,1]) to prevent features with large magnitudes from dominating, improving model training efficiency.



Normalization (Min-Max Scaling)

- Min-max scaling (or normalization) is a data preprocessing technique that transforms numerical features into a specific, bounded range—usually 0 to 1—by mapping the minimum value to 0 and the maximum value to 1.
- It ensure all features contribute equally to models, particularly for algorithms sensitive to magnitude, such as neural networks and distance-based methods.
- preserve the original distribution shape

$$X_{\text{scaled}} = \frac{X_i - X_{\min}}{X_{\max} - X_{\min}}$$



Normalization (Min-Max Scaling)

- **Outlier Sensitivity:** Highly sensitive to outliers; a single extreme outlier can compress normal data into a tiny range (e.g., 0.0 to 0.1).
- **Application:** Best used when data does not follow a Gaussian distribution, such as in image processing (pixel intensities 0-255) or neural networks.
- **Pros and Cons**
 - Pros: Preserves relationships among original data points; fits data into a strict range.
 - Cons: Not robust to outliers; reduces standard deviation, which can hide the impact of outliers.



Example Code

```
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# Sample Data: [Age, Experience]
# Sample data with different scales
# Feature 1 (e.g., Age) has values from 0 to 11
# Feature 2 (e.g., Experience) has values from 2 to 10
data = np.array([[11, 2],
                 [3, 7],
                 [0, 10],
                 [11, 8]])

# 1. Create an instance of the MinMaxScaler
scaler = MinMaxScaler()

# 2. Fit the scaler to the data and transform the data
scaled_data = scaler.fit_transform(data)

# 3. Print the original and scaled data
print("Original Data:\n", data)
print("\nScaled Data (between 0 and 1):\n", scaled_data)
# Output: Scaled values between 0 and 1
```

Original Data:

```
[[11  2]
 [ 3  7]
 [ 0 10]
 [11  8]]
```

Scaled Data (between 0 and 1):

```
[[1.         0.         ]
 [0.27272727 0.625     ]
 [0.         1.         ]
 [1.         0.75      ]]
```



```

# MinMaxScaler feature_range=(-1,1)
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# Sample Data: [Age, Experience]
# Sample data with different scales
# Feature 1 (e.g., Age) has values from 0 to 11
# Feature 2 (e.g., Experience) has values from 2 to 10
data = np.array([[11, 2],
                 [3, 7],
                 [0, 10],
                 [11, 8]])

# 1. Create an instance of the MinMaxScaler
scaler = MinMaxScaler(feature_range=(-1,1))

# 2. Fit the scaler to the data and transform the data
scaled_data = scaler.fit_transform(data)

# 3. Print the original and scaled data
print("Original Data:\n", data)
print("\nScaled Data (between 0 and 1):\n", scaled_data)
# Output: Scaled values between 0 and 1

```

Original Data:

```

[[11  2]
 [ 3  7]
 [ 0 10]
 [11  8]]

```

Scaled Data (between 0 and 1):

```

[[ 1.         -1.         ]
 [-0.45454545  0.25        ]
 [-1.         1.         ]
 [ 1.         0.5         ]]

```



Example 2: Normalization

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
# fit the scaler to the train set, it will learn the parameters  
scaler.fit(X_train)
```

```
# transform train and test sets
```

```
X_train_scaled1 = scaler.transform(X_train)
```

```
X_test_scaled1 = scaler.transform(X_test)
```

```
X_train_scaled1 = pd.DataFrame(X_train_scaled1, columns=X_train.columns)
```

```
X_test_scaled1 = pd.DataFrame(X_test_scaled1, columns=X_test.columns)
```

```
X_train.describe(), X_train_scaled1.describe()
```

```
14]: (
      count      Age  EstimatedSalary
count  280.000000    280.000000
mean    37.864286   69807.142857
std     10.218201   34641.201654
min     18.000000   15000.000000
25%     30.000000   43000.000000
50%     37.000000   70500.000000
75%     46.000000   88000.000000
max     60.000000  150000.000000,

      count      Age  EstimatedSalary
count  280.000000    280.000000
mean     0.472959     0.405979
std     0.243290     0.256601
min     0.000000     0.000000
25%     0.285714     0.207407
50%     0.452381     0.411111
75%     0.666667     0.540741
max     1.000000     1.000000)
```



D Y PATIL
UNIVERSITY

NAVI MUMBAI

Feature scaling

Normalization

Standardization

$$X_{new} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

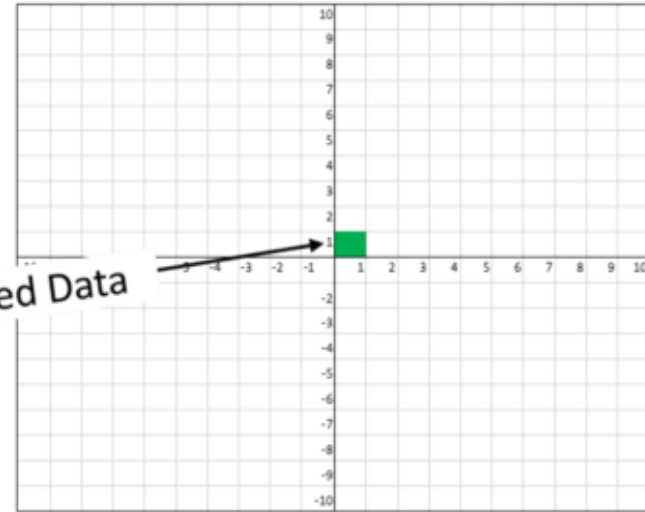
$$X' = \frac{X - \text{Mean}}{\text{Standard deviation}}$$



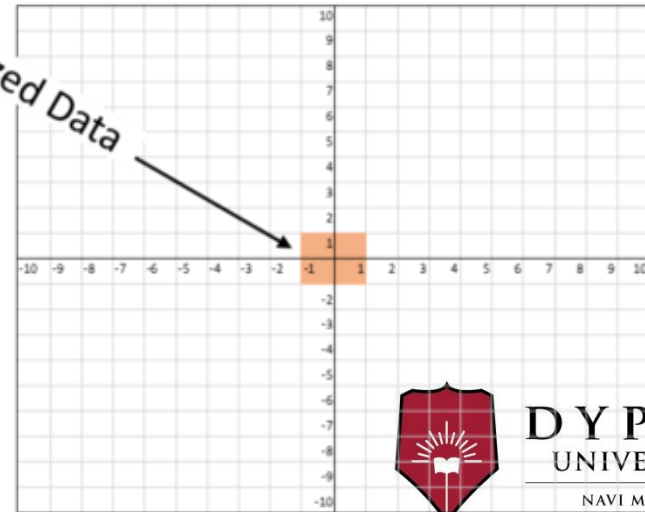
Feature Scaling: Normalization and Standardization



Normalized Data



Standardized Data



D Y PATIL
UNIVERSITY

NAVI MUMBAI

Aspect	Normalization	Standardization
Objective	Rescale data to a fixed range (usually 0 to 1).	Rescale data to have a mean of 0 and a standard deviation of 1.
Formula	$X_{\text{norm}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$	$Z = (X - \mu) / \sigma$
Range of Transformed Data	[0, 1] or [-1, 1] (depending on the technique).	Data is centered around 0 with a standard deviation of 1.
Sensitivity to Outliers	Sensitive to outliers, as they affect the min and max values.	Less sensitive to outliers because the transformation uses mean and standard deviation.
Preferred Algorithms	Distance-based algorithms (e.g., k-NN, k-means, neural networks).	Algorithms that assume normally distributed data (e.g., linear regression, PCA, SVM).
When to Use	When features have known min and max values and when the model is distance-based.	When features have different scales, contain outliers, or the model assumes normally distributed data.



When to use Normalization or Standardization

- Normalization (min-max scaling to 0–1) is best when data does not follow a Gaussian distribution, or when algorithms require bounded ranges (e.g., neural networks, image processing, KNN).
- Standardization (z-score scaling to mean=0, std=1) is ideal for algorithms assuming Gaussian distribution, managing outliers, and algorithms calculating distances (e.g., SVM, PCA, Linear Regression).



Normalization (Min-Max Scaling)

- Normalization (Min-Max Scaling)
 - Where to Use:
 - Image Processing: Pixel intensities (0–255) to 0–1 range.
 - Algorithms: Neural Networks (faster convergence), K-Nearest Neighbors (KNN).
 - Data Characteristics: When the data distribution is not Gaussian.
 - Boundary Sensitive: When you need a strictly bounded range (e.g., 0 to 1 or -1 to 1).



Standardization (Z-Score Normalization)

- Where to Use:
 - ✓ – Algorithms: Support Vector Machines (SVM), Principal Component Analysis (PCA), Linear/Logistic Regression.
 - ✓ – Data Characteristics: When the data follows a Gaussian (normal) distribution.
 - ✓ – Outliers: When the data contains outliers, as it is less sensitive to extreme values.
 - ✓ – Clustering: Algorithms that calculate distances (e.g., K-Means).



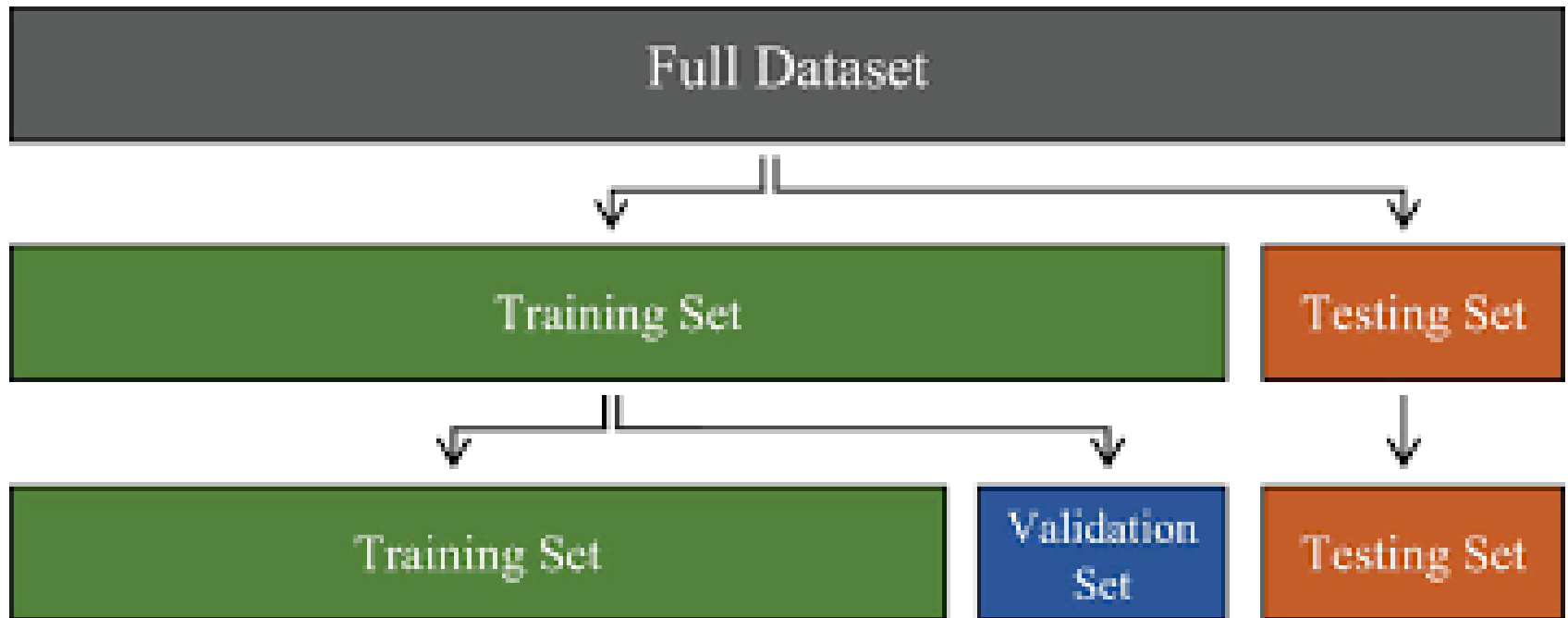
Dataset splitting strategies:

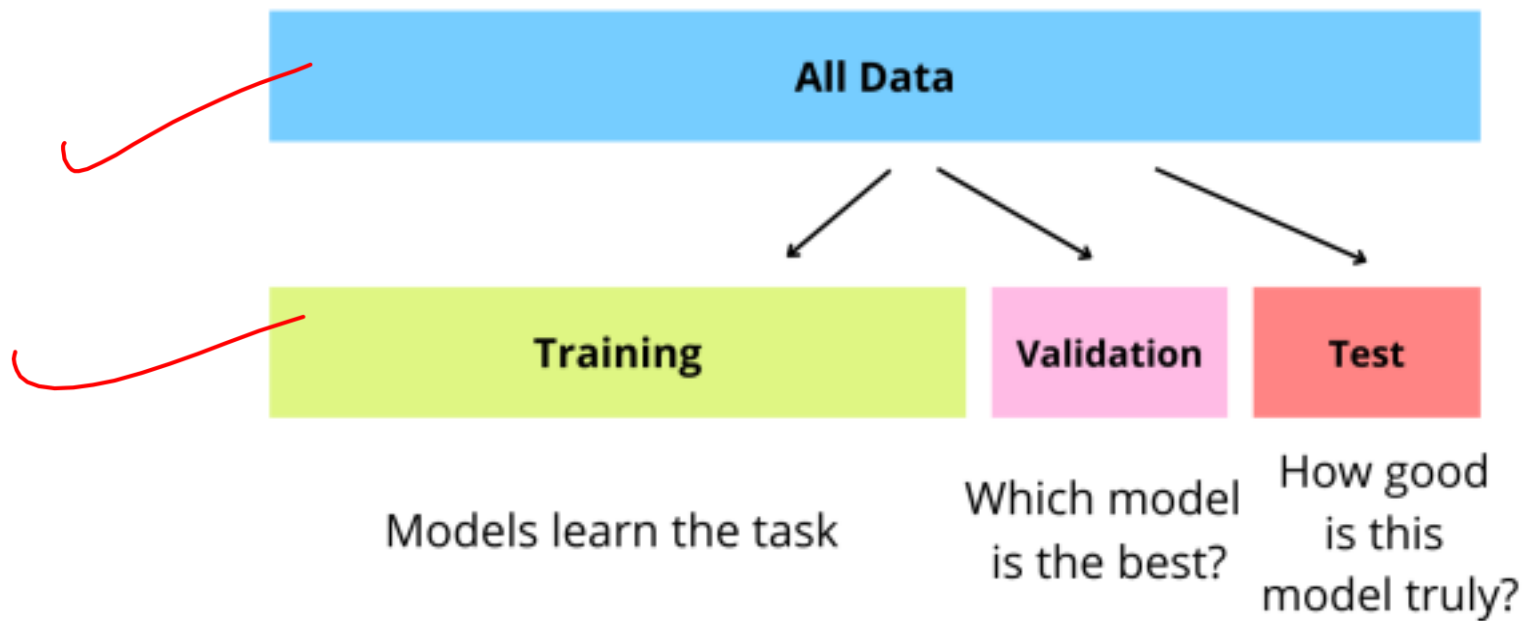
Training, validation, and test sets

- Dataset splitting involves partitioning a dataset into subsets to train, tune, and evaluate a machine learning model effectively.
- Data splitting is the process of splitting data into 3 sets:
 - Data which we use to design our models (**Training set**)
 - Data which we use to refine our models (**Validation set**)
 - Data which we use to test our models (**Testing set**)
- Training proceeds on the training set, after which evaluation is done on the validation set, and when the experiment seems to be successful, final evaluation can be done on the test set.



Dataset splitting strategies: Training, validation, and test sets





Dataset splitting strategies: Training, validation, and test sets

- **Training Set**

- **Purpose:** This is the largest portion of the data used to fit the model and optimize its internal parameters (e.g., the weights in a neural network). The model learns the underlying patterns and relationships from this data.
- **Role:** It provides the foundation for the model's "knowledge". If the training set is too small or not diverse, the model may not learn effectively.



Dataset splitting strategies: Training, validation, and test sets

- **Validation Set**
 - **Purpose:** Used during the model development and training process to tune the model's hyperparameters (e.g., value of k in KNN, learning rate) and perform model selection.
 - **Role:** It acts as a checkpoint to provide an unbiased evaluation of the model's performance on new data during training, helping to detect and prevent **overfitting** (when a model performs well on training data but poorly on unseen data).
 - It should not overlap with the training or test sets.



Dataset splitting strategies: Training, validation, and test sets

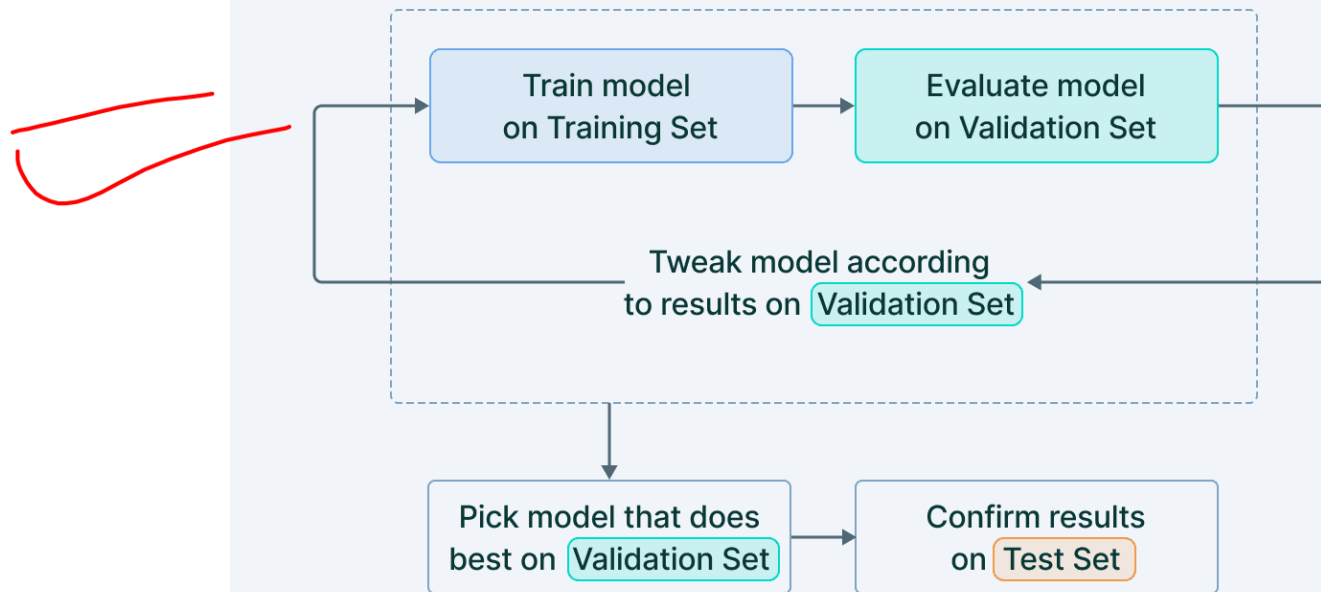
- **Test Set**

- **Purpose:** A completely held-out, untouched portion of the dataset used only once at the very end to provide a final, unbiased estimate of the model's performance on real-world, unseen data.
- **Role:** It measures the model's true generalization ability. The results from the test set are the most reliable indicator of how the model will perform in production.



Dataset splitting strategies: Training, validation, and test sets

Training data/validation/test



V7 Labs



D Y PATIL
UNIVERSITY
NAVI MUMBAI

The Three Data Subsets

Subset 	Purpose	Key Outcome
<u>Training Set</u>	Used to fit the model and learn hidden patterns or parameters.	A model that maps inputs to outputs.
<u>Validation Set</u>	Used to tune hyperparameters (e.g., learning rate) and make design decisions during development.	Selection of the best model configuration; detects overfitting early.
<u>Test Set</u>	A "final exam" used only once to provide an unbiased estimate of real-world performance.	Unbiased final accuracy and generalization metrics.



Dataset splitting strategies: Training, validation, and test sets

- **Typical Ratios:** Common split ratios include 70% train/15% validation/15% test, or 80% train/10% validation/10% test. For very large datasets, smaller validation and test sets (e.g., 98/1/1) are often sufficient.



Cross Validation

- By partitioning the available data into three sets, we drastically reduce the number of samples which can be used for learning the model, and the results can depend on a particular random choice for the pair of (train, validation) sets.
- A solution to this problem is a procedure called cross-validation (CV for short).
- A test set should still be held out for final evaluation, but the validation set is no longer needed when doing CV.

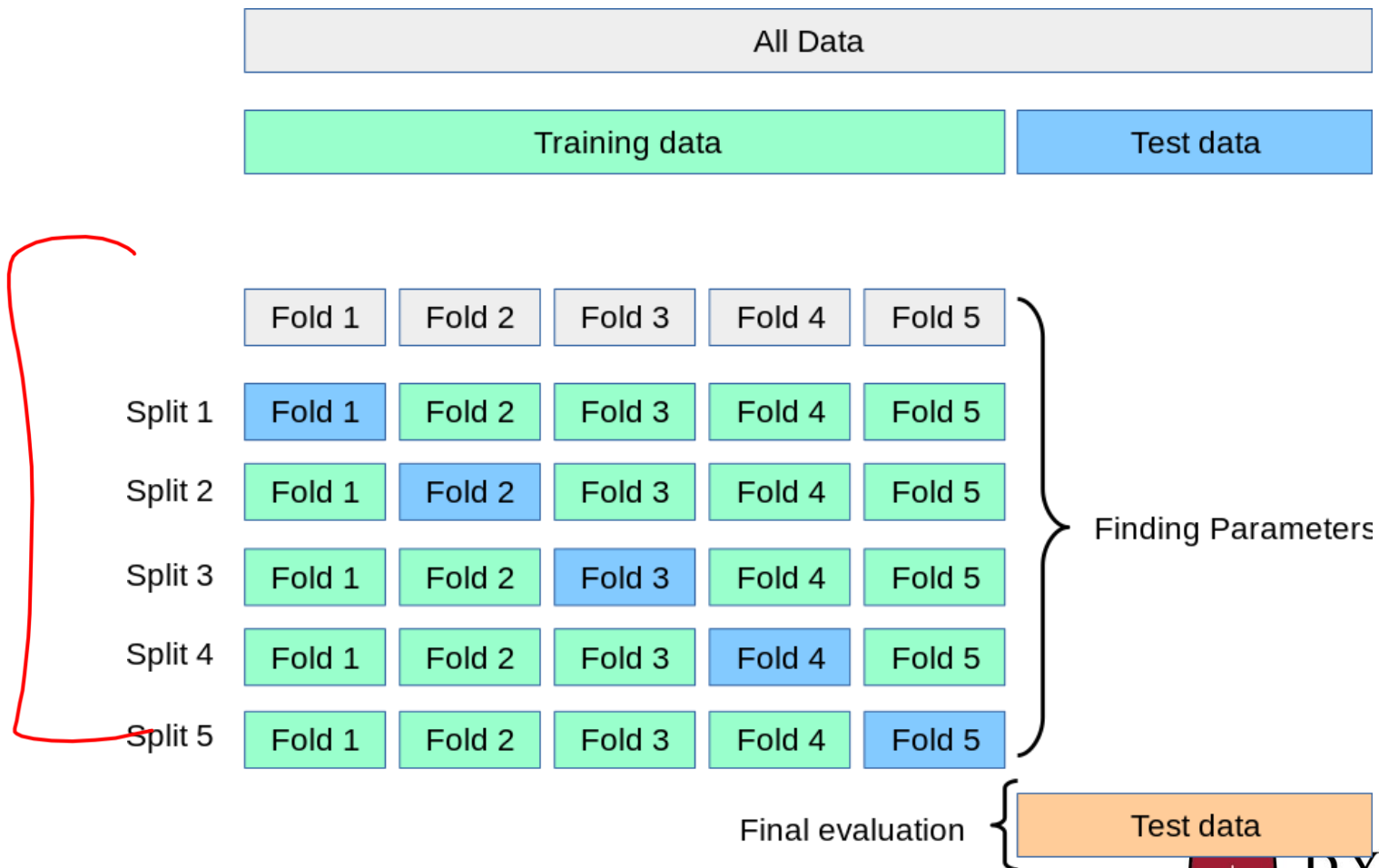


K fold Cross Validation Procedure

- In the basic approach, called k -fold CV, the training set is split into k smaller sets.
- The following procedure is followed for each of the k “folds”:
 - A model is trained using $k-1$ of the folds as training data;
 - the resulting model is validated on the remaining part of the data (i.e., it is used as a test set to compute a performance measure such as accuracy).
- The performance measure reported by k -fold cross-validation is then the average of the values computed in the loop.
- This approach can be computationally expensive, but does not waste too much data (as is the case when fixing an arbitrary validation set).



K Fold Cross Validation



K Fold Cross Validation

